计算机系统 I：处理器基本原理与设计 (The Processor) 复习讲义

目录

1 计算机体系结构概览 (Computer Architecture Overview) 2
1.1 层次结构 (System Stack) . . . 2
1.2 冯 · 诺依曼结构 (Von Neumann Structure) . . . 2
1.3 多核处理器示例 (Multicore Example: AMD Barcelona) . . . 2

2 CPU 内部结构 (Inside the Processor) 3
2.1 CPU 的三大核心部件 (Datapath, Control & Cache) . . . 3
2.2 三种设计方法论 (Three Design Methodologies) . . . 3
2.3 可编程与非可编程系统 (Programmable vs Non-programmable) . . . 4
2.4 CPU 的本质 (CPU as a General Digital System) . . . 4

3 CPU 关键部件详解 (Key CPU Components) 4
3.1 算术逻辑运算单元 (ALU) . . . 4
3.2 存储器 (Memory) . . . 4
3.3 寄存器 (Register) . . . 5
3.4 寄存器文件 (Register File) . . . 5
3.5 立即数生成单元 (Immediate Generation Unit, ImmGen) . . . 6
3.6 RISC-V 寄存器约定 (RISC-V Register Conventions) . . . 6
3.7 RISC-V 指令字段 (RISC-V Instruction Fields) . . . 7
3.8 RISC-V 汇编语言概览 (RISC-V Assembly Language) . . . 7
3.9 指令编码示例 (Instruction Encoding Examples) . . . 7
3.10 逻辑设计约定 (Logic Design Conventions) . . . 7

4 CPU 性能分析 (CPU Performance Analysis) 8
4.1 基本性能指标 (Key Metrics) . . . 8
4.2 CPU 时钟与时间公式 (CPU Clock & Time Formula) . . . 8

5 性能提升与三堵墙 (Performance Improvement & Three Walls) **10**
5.1 性能改进方向 (Performance Improvement Directions) 10
5.2 三堵墙 (Three Walls) 10

6 多处理器与 Amdahl 定律 (Multiprocessors & Amdahl's Law) **11**
6.1 多处理器 (Multiprocessors) 11
6.2 Amdahl 定律 (Amdahl's Law) 11

7 构建数据通路 (Building the Datapath) **13**
7.1 本课程简化 RISC-V CPU 所支持的指令集 13
7.2 指令执行的五个阶段 (Five Stages of Instruction Execution) 13
7.3 取指单元 (Instruction Fetch Unit) 13
7.4 各类型指令的数据流 (Data Paths by Instruction Type) 14
7.5 组合数据通路的原则 (Composing the Elements) 15
7.6 完整单周期数据通路 (Full Single-Cycle Datapath) 16

8 控制器设计 (Controller Design) **16**
8.1 控制器的设计思路 (Design Approach) 16
8.2 控制信号总览 (Control Signals Overview) 16
8.3 两级译码结构 (Two-Level Decoder) 17
8.4 各指令的控制信号真值表 (Control Signals Truth Table) 18
8.5 控制器 Verilog 实现 (Controller Verilog Implementation) 18

9 单周期实现的性能分析 (Single-Cycle Implementation Performance) **20**
9.1 关键路径分析 (Critical Path Analysis) 20
9.2 单周期实现的缺陷 (Limitations of Single-Cycle) 20

10 全章总结 (Summary) **21**

1 计算机体系结构概览 (Computer Architecture Overview)

1.1 层次结构 (System Stack)

计算机系统是由多个抽象层次构成的栈式体系结构，ISA 处于软硬件分界面：

层次	说明
Problem	用户待解决的问题
Algorithm	解决问题的算法
Program	程序（高级语言）
Runtime System (VM/OS/MM)	运行时环境：虚拟机、操作系统、内存管理
ISA (Architecture)	**软硬件接口**：指令集体系结构
Microarchitecture	微体系结构：硬件具体实现方式
Logic / Circuits	逻辑门与电路
Electrons	物理层：电子信号

1.2 冯 · 诺依曼结构 (Von Neumann Structure)

冯 · 诺依曼结构核心原则

存储程序计算机 (Stored-Program Computer)：程序和数据都存储在同一存储器中。CPU 从存储器取指令和数据，执行运算后将结果写回存储器。

冯 · 诺依曼结构的三大核心组成：

- **中央处理器 (CPU)**：负责取指、译码、执行。
- **存储器 (Memory)**：存放指令和数据，按地址访问。
- **输入输出 (I/O)**：与外部设备交互。

1.3 多核处理器示例 (Multicore Example: AMD Barcelona)

AMD Barcelona 是早期多核处理器的代表，在单块芯片上集成了 **4 个处理器核心**，每个核心拥有独立的 L1/L2 Cache，并共享 L3 Cache。这是应对” 三堵墙”、从单核转向多核时代的典型案例。

2 CPU 内部结构 (Inside the Processor)

2.1 CPU 的三大核心部件 (Datapath, Control & Cache)

CPU 三大核心部件

- **数据通路 (Datapath)**：对数据执行算术和逻辑操作的部件集合，包括寄存器、ALU、多路选择器 (MUX)、存储器等。
- **控制器 (Control Unit)**：根据指令指挥数据通路、存储器和 I/O 设备按序完成操作，产生控制信号。
- **高速缓存 (Cache)**：小而快的 SRAM，位于 CPU 芯片内部，缓存频繁访问的指令和数据，弥补 CPU 与主存之间的速度差异。

2.2 三种设计方法论 (Three Design Methodologies)

方法一：有限状态机 (Finite State Machine, FSM)

将数字系统建模为一组状态和状态转移函数，适用于**非可编程专用系统**(Non-programmable System)。控制单元直接编码所有状态转移逻辑，不涉及取指和执行指令。

方法二：寄存器传输控制 (Register Transfer Language, RTL)

通过描述寄存器间的数据传输来设计系统，三要素：

1. **寄存器集合 (Set of Registers)**：大部分在数据通路中，部分在控制单元中。
2. **基本操作 (Micromanipulation)**：寄存器间传输的原子操作。
3. **控制 (Control)**：监督和调度寄存器传输的执行顺序。

方法三：程序状态机 (Program State Machine, PSM)

通用系统 (General System) 的设计方法。输入中包含一段存储在存储器中、由程序计数器 (PC) 寻址的**指令序列**。控制单元负责取指、译码并执行指令，是现代 CPU 的核心设计范式。

PSM 的核心信号流：

- **控制信号 (Control Signals)**：控制单元输出，驱动数据通路。
- **状态信号 (Status Signals)**：数据通路输出给控制单元，用于条件判断（如 Zero 信号）。
- **控制输入/输出 (Control Input/Output)**：与外部 I/O 设备的控制交互。
- **数据输入/输出 (Data Input/Output)**：与存储器和 I/O 的数据交互。

2.3 可编程与非可编程系统 (Programmable vs Non-programmable)

非可编程系统 (Specific System)	**可编程系统 (General System / CPU)**
控制单元不取指/执行指令	控制单元从 PC 指向的存储器取指并执行
包含所有状态转移逻辑（基于输入和状态位）	通过程序灵活控制，功能通用
FSM 实现	PSM 实现（即 CPU）

2.4 CPU 的本质 (CPU as a General Digital System)

CPU 是一个**通用数字系统**，本质上是一台图灵机 (Turing Machine)，由寄存器传输控制技术实现：

- **数据通路 (Datapath)**：执行算术运算的部件。
- **控制器 (Control)**：根据程序指令指挥数据通路、存储器和 I/O。

3 CPU 关键部件详解 (Key CPU Components)

3.1 算术逻辑运算单元 (ALU)

ALU 是执行算术和逻辑运算的核心部件。在本课程的简化 RISC-V CPU 中，ALU 支持以下 5 种基本操作（由 3 位 ALU 控制信号决定）：

ALU 控制信号	**操作**	**说明**
000	AND	按位与
001	OR	按位或
010	Add	加法
110	Sub	减法
111	Slt	Set on less than：若 $A < B$ 则结果为 1，否则为 0

ALU 还输出 **Zero** 信号，用于分支指令（如 `beq`）的条件判断：当 ALU 执行减法后结果为 0，则 Zero=1，表示两操作数相等。

3.2 存储器 (Memory)

CPU 中涉及两类存储器：

	指令存储器 (Instruction Memory)	**数据存储器 (Data Memory)**
访问方式	只读 (Read-Only)	可读写
控制信号	无（地址输入后直接输出指令）	MemRead, MemWrite
作用	根据 PC 输出当前指令	执行 `lw`/`ld` (读) 和 `sw`/`sd` (写)

3.3 寄存器 (Register)

单个寄存器是一个**时序逻辑单元 (State Element)**，由 D 触发器构成，受 Write 信号控制：

- **Write = 0**：数据输出保持原状态不变。
- **Write = 1**：在有效时钟边沿（上升沿），数据输出 = 数据输入。

3.4 寄存器文件 (Register File)

寄存器文件由 **32 个 64 位寄存器** (x0–x31) 构成，x0 硬连线为 0（写入无效）。

- **输入**：两个 5 位读地址 (Rs1_addr, Rs2_addr)，一个 5 位写地址 (Wt_addr)，一个 64 位写数据 (Wt_data)，以及写控制信号 RegWrite。
- **输出**：两个 64 位读数据 (Rs1_data, Rs2_data)。
- **时序**：读是组合逻辑（异步立即输出）；写在时钟**上升沿**触发，且仅当 RegWrite=1 时有效。

寄存器文件的 Verilog 模块接口（32×64 bits）：

```
module RegFile(
    input  clk, rst, RegWrite,
    input  [4:0]  Rs1_addr, Rs2_addr, Wt_addr,
    input  [63:0] Wt_data,
    output [63:0] Rs1_data, Rs2_data
);
    reg [63:0] register [1:31];   // r1 - r31
    assign Rs1_data = (Rs1_addr == 0) ? 0 : register[Rs1_addr];
    assign Rs2_data = (Rs2_addr == 0) ? 0 : register[Rs2_addr];
    always @(posedge clk or posedge rst) begin
        if (rst) for (i=1; i<32; i=i+1) register[i] <= 0;
        else if ((Wt_addr != 0) && (RegWrite == 1))
```

```
            register[Wt_addr] <= Wt_data;
    end
endmodule
```

3.5 立即数生成单元 (Immediate Generation Unit, ImmGen)

ImmGen 根据指令类型从 32 位指令中提取并**符号扩展**为 64 位立即数，由 **ImmSel** 控制信号选择类型：

ImmSel	类型	典型指令	立即数位字段（Verilog 风格）
000	I-type	`lw/ld, addi`	`{{52{inst[31]}}, inst[31:20]}`
001	S-type	`sw/sd`	`{{52{inst[31]}}, inst[31:25], inst[11:7]}`
010	SB-type	`beq, bne`	`{{51{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0}`
011	UJ-type	`jal`	`{{43{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0}`

注意 SB 和 UJ 型指令的立即数末位硬编码为 0，即偏移量始终为**偶数**（字节对齐）。另外，S 型指令立即数散布于指令字的两个字段（inst[31:25] 和 inst[11:7]），ImmGen 负责拼接。

3.6 RISC-V 寄存器约定 (RISC-V Register Conventions)

RISC-V 共有 32 个 64 位通用寄存器 (x0–x31)，各有固定用途约定：

寄存器	别名	用途
x0	zero	常量 0（写入无效）
x1	ra	返回地址 (Return Address)
x2	sp	栈指针 (Stack Pointer)
x3	gp	全局指针 (Global Pointer)
x4	tp	线程指针 (Thread Pointer)
x5–x7	t0–t2	临时寄存器 (Temporaries)
x8–x9	s0–s1	保存寄存器 (Saved)
x10–x17	a0–a7	函数参数/返回值 (Arguments/Results)
x18–x27	s2–s11	保存寄存器 (Saved)
x28–x31	t3–t6	临时寄存器 (Temporaries)

3.7 RISC-V 指令字段 (RISC-V Instruction Fields)

每条 RISC-V 指令为 32 位，包含以下字段：

- **opcode**（7 位）：指令的基本操作类型。
- **rs1**（5 位）：第一个源寄存器编号。
- **rs2**（5 位）：第二个源寄存器编号。
- **rd**（5 位）：目标寄存器编号。
- **funct3**（3 位）：功能字段，区分同 opcode 下的不同操作。
- **funct7**（7 位）：扩展功能字段（R 型指令）。
- **immediate**：立即数/地址偏移量（位置因指令类型而异）。

3.8 RISC-V 汇编语言概览 (RISC-V Assembly Language)

类别	指令	功能
算术 (Arithmetic)	`add, sub, addi`	加法、减法、加立即数
数据传输 (Data Transfer)	`ld, sd`	加载/存储双字
逻辑 (Logical)	`and, or, xor, srl`	按位与/或/异或/逻辑右移
条件分支 (Conditional Branch)	`beq, bne`	相等/不等时跳转
无条件跳转 (Unconditional Branch)	`jal, jalr`	跳转并链接

3.9 指令编码示例 (Instruction Encoding Examples)

- **SB-type：`bne x10, x11, 2000`**：若 x10 $\neq$ x11，跳转至 PC+2000。立即数 2000 被编码为 SB 格式（末位补 0，各位散布于指令字中）。
- **UJ-type：`jal x0, 2000`**：无条件跳转至 PC+2000，返回地址写入 x0（即丢弃）。立即数 2000 编码为 UJ 格式（末位补 0）。

3.10 逻辑设计约定 (Logic Design Conventions)

- 信息用二进制编码，低电压 = 0，高电压 = 1；每条线传一位，多位数据用总线 (Bus) 传输。
- **组合逻辑单元 (Combinational Element)**：输出是输入的纯函数，无内部状态（如 ALU、MUX）。
- **时序逻辑单元 (State Element)**：具有内部存储状态（如寄存器、存储器），受时钟控制。

- **边沿触发方法 (Edge Triggered Methodology)**：在时钟**上升沿**采样并写入状态单元。

典型执行周期：

1. 读取状态单元的输出（寄存器、PC 等）。
2. 数据流经组合逻辑（ALU、MUX、ImmGen 等）进行运算。
3. 在下一时钟上升沿，将结果写入状态单元（寄存器文件、PC、数据存储器）。

4 CPU 性能分析 (CPU Performance Analysis)

4.1 基本性能指标 (Key Metrics)

响应时间与吞吐量 (Latency & Throughput)

- **响应时间/延迟 (Latency / Response Time)**：完成一项任务所需的总时间，是单用户最关心的指标。
- **吞吐量 (Throughput / Bandwidth)**：单位时间内完成的任务总量（如每小时处理的事务数）。

性能定义：$\text{Performance}_X = \dfrac{1}{\text{Execution Time}_X}$

"X 比 Y 快 n 倍" 等价于：

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = n$$

示例：计算机 A 执行某程序需 10s，计算机 B 需 15s，则 A 比 B 快 $15/10 = 1.5$ 倍。

CPU 时间 (CPU Time)

- **经过时间 (Elapsed Time)**：包括 I/O、OS 开销、等待等所有时间，反映系统整体性能。
- **CPU 时间 (CPU Time)**：仅包含 CPU 处理该程序的时间，分为用户 CPU 时间（执行程序本身）和系统 CPU 时间（OS 代表程序执行）。

4.2 CPU 时钟与时间公式 (CPU Clock & Time Formula)

CPU 性能公式（必须掌握）

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Period} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}} \\ &= \text{Instruction Count} \times \text{CPI} \times \text{Clock Period} \\ &= \frac{\text{IC} \times \text{CPI}}{\text{Clock Rate (Hz)}}\end{aligned}$$

其中：

- **时钟周期 (Clock Period)**：一个时钟周期的时长，如 $250\,\text{ps} = 0.25\,\text{ns}$。
- **时钟频率 (Clock Rate)**：每秒周期数，如 $4.0\,\text{GHz} = 4 \times 10^9\,\text{Hz}$。两者互为倒数。
- **指令数 (Instruction Count, IC)**：由程序、ISA 和编译器决定，与硬件无关。
- **每指令周期数 (Cycles Per Instruction, CPI)**：由 CPU 硬件决定；若不同指令的 CPI 不同，则需加权平均。

性能改进方向：减少时钟周期数，或提高时钟频率——两者常相互制约，设计者需权衡。

CPU 时间例题

例：计算机 A 时钟频率 2 GHz，执行某程序 CPU 时间为 10s。计算机 B 目标 CPU 时间为 6s，且时钟周期数为 A 的 1.2 倍，求 B 的时钟频率。

$$\text{Clock Cycles}_A = 2\,\text{GHz} \times 10\,\text{s} = 20 \times 10^9$$

$$\text{Clock Cycles}_B = 1.2 \times 20 \times 10^9 = 24 \times 10^9$$

$$\text{Clock Rate}_B = \frac{24 \times 10^9}{6\,\text{s}} = 4\,\text{GHz}$$

CPI 比较例题

例：计算机 A 时钟周期 250ps，CPI=2.0；计算机 B 时钟周期 500ps，CPI=1.2。哪台更快？快多少？

$$\text{CPU Time}_A = \text{IC} \times 2.0 \times 250\,\text{ps} = 500 \cdot \text{IC}\,\text{ps}$$

$$\text{CPU Time}_B = \text{IC} \times 1.2 \times 500\,\text{ps} = 600 \cdot \text{IC}\,\text{ps}$$

A 比 B 快 $600/500 = 1.2$ 倍。

加权平均 CPI (Weighted Average CPI)

$$\text{CPI} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}} \right) = \sum_{i=1}^{n} \text{CPI}_i \times \text{Relative Frequency}_i$$

例：三类指令 A、B、C，CPI 分别为 1、2、3：

指令类	CPI	序列 1 IC	序列 2 IC
A	1	2	4
B	2	1	1
C	3	2	1
合计	–	IC=5, 周期 =10, 均 CPI=2.0	IC=6, 周期 =9, 均 CPI=1.5

序列 2 虽然指令数更多，但平均 CPI 更低，总时钟周期数更少，因此执行更快。

性能影响因素汇总

因素	**影响 IC**	**影响 CPI**	**影响 T_c**
算法 (Algorithm)	✓	✓	
编程语言 (Language)	✓	✓	
编译器 (Compiler)	✓	✓	
ISA	✓	✓	✓

5 性能提升与三堵墙 (Performance Improvement & Three Walls)

5.1 性能改进方向 (Performance Improvement Directions)

单处理器性能的提升主要从以下几个方向入手：

- **I/O 发展**：提升输入输出设备的速度。
- **存储器组织结构**：联想存储器、通用寄存器组、Cache 等。
- **指令集发展**：CISC（复杂指令集）→ RISC（精简指令集）。
- **并行处理技术**：微操作级、指令级、线程级、进程级、任务级并行。

5.2 三堵墙 (Three Walls)

单处理器性能的提升遭遇了三大瓶颈，即” 三堵墙”：

三堵墙 (Three Walls)

1. **功耗墙 (Power Wall)**
   CMOS 功耗公式：$P \propto C \cdot V^2 \cdot f$（C 为电容，V 为电压，f 为频率）。
   频率翻倍则功耗翻倍，产生严重热点 (Hot-spot) 问题；漏电流 (Power Leakage) 也随之增加。
   功耗从 1980s 到 2000s 增加约 **1000 倍**；电压从 5V 降至约 1V，已难以继续降低；散热能力达到极限。

   功耗降低示例：假设新 CPU 电容负载为原来的 85%，电压和频率各降低 15%，则：

   $$P_{\text{new}} = 0.85C \times (0.85V)^2 \times 0.85f = 0.85^4 \times P_{\text{old}} \approx 0.52\, P_{\text{old}}$$

   即使如此，功耗墙仍无法突破——继续降低电压会影响电路稳定性，继续提高频率会使散热无法维持。

2. **存储墙 (Memory Wall)**
   CPU 速度提升远快于 DRAM 主存，延迟差距持续扩大。
   1980 年微处理器无片上 Cache；1989 年 Intel 首次集成 Cache；2001 年已有两级片上 Cache。
   Cache 增大带来的性能边际收益递减（L2 Cache 越来越大，但贡献越来越小）。

3. **指令级并行墙 (ILP Wall)**
   单进程指令流中可挖掘的 ILP 已趋于枯竭，仅约 **20%** 的 ILP 剩余可被开发。
   应对方向：从 ILP 转向线程级并行 (TLP) 和数据级并行 (DLP)，即多核处理器。

6 多处理器与 Amdahl 定律 (Multiprocessors & Amdahl's Law)

6.1 多处理器 (Multiprocessors)

多核微处理器 (Multicore Microprocessors) 是应对三堵墙的主要方案：在一块芯片上集成多个处理器核心，要求程序通过**显式并行编程**来利用多核优势。

- 与 ILP（对程序员透明，由硬件自动挖掘）不同，多核并行需要程序员**显式编写并行代码**。
- 主要挑战：**负载均衡 (Load Balancing)**、**通信与同步的开销优化 (Communication & Synchronization)**。

6.2 Amdahl 定律 (Amdahl's Law)

基本公式推导

Amdahl 定律的出发点：

$$T_{\text{new}} = T_{\text{cannot enhance}} + \frac{T_{\text{enhanced}}}{S_e}$$

设 F_e 为改进部分占总时间的比例，S_e 为该部分自身的加速比，则：

Amdahl 定律公式

$$\text{Speedup}_{\text{overall}} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}}$$

其中：

- F_e（**改进比例 / Fraction Enhanced**）：改进部分占系统总执行时间的比例（$0 \leq F_e \leq 1$）。
  例如：一个程序共需 60s，其中 20s 可被加速，则 $F_e = 20/60$。
- S_e（**部件加速比 / Speedup Enhanced**）：改进部分自身的加速倍数，即改进前与改进后该部分执行时间之比。
  例如：改进后该部分从 5s 降至 2s，则 $S_e = 5/2 = 2.5$。

Amdahl 定律的关键推论

性能改进递减规律与上界推论

1. **性能改进递减规律 (Decreasing Rule)**：若只改进一部分计算任务，改进程度越高，对整体性能的提升就越受限。
2. **加速比上界 (Upper Bound)**：即使将改进部分的速度提升到无限快（$S_e \to \infty$），整体加速比不超过：
   $$\text{Speedup}_{\max} = \frac{1}{1 - F_e}$$
3. **实践意义**：优化应**优先针对执行时间占比最大的部分**（即 **Make the Common Case Fast**）。

" 无法达到" 的反例 (Counter-example: "Can't be done")

例：乘法运算占程序总运行时间的 80s/100s（即 $F_e = 80\%$）。想通过改进乘法性能使**整体**性能提升 5 倍，需要将乘法加速多少倍?

$$5 = \frac{1}{(1 - 0.8) + \frac{0.8}{S_e}} \Rightarrow 0.2 + \frac{0.8}{S_e} = 0.2 \Rightarrow \frac{0.8}{S_e} = 0 \Rightarrow S_e \to \infty$$

结论：这是不可能实现的！即使将乘法运算的速度提升到无限快，整体加速比上界也只有 $1/(1 - 0.8) = 5$ 倍，这要求非乘法部分的时间趋近于零——实际上无法达到。

典型例题

例 1：某功能的处理速度提升为原来的 20 倍，但该功能仅占系统总运行时间的 40%，整体性能提升为多少?

$$\text{Speedup} = \frac{1}{(1-0.4)+\frac{0.4}{20}} = \frac{1}{0.6+0.02} = \frac{1}{0.62} \approx 1.613 \text{ 倍}$$

例 2：采用浮点运算部件后，浮点运算速度提升 20 倍，整体性能提升 5 倍。求浮点运算占该程序的比例。

$$5 = \frac{1}{(1-F_e)+\frac{F_e}{20}} \Rightarrow F_e \approx 84.2\%$$

7 构建数据通路 (Building the Datapath)

7.1 本课程简化 RISC-V CPU 所支持的指令集

类型	指令	功能
访存 (Memory-reference)	`lw/ld`, `sw/sd`	加载/存储
算术逻辑 (Arithmetic-logical)	`add, sub, and, or, slt, addi, xor, srl`	R/I 型运算
控制流 (Control Flow)	`beq, bne, jal, jalr`	分支与跳转

7.2 指令执行的五个阶段 (Five Stages of Instruction Execution)

RISC-V 指令执行的五个阶段（单周期 CPU）

1. **取指 (IF, Instruction Fetch)**：从指令存储器读出 PC 指向的指令，PC ← PC+4。
2. **译码/读操作数 (ID, Instruction Decode)**：对指令译码，读取寄存器文件中的操作数（无论是否用到，都读出备用）。
3. **执行 (EX, Execute)**：ALU 执行计算——R 型做算术/逻辑运算；访存指令计算地址；分支指令做比较；跳转指令计算目标地址。
4. **访存 (MEM, Memory Access)**：仅 `lw/ld`（Load）和 `sw/sd`（Store）指令使用此阶段，其余指令直通。
5. **写回 (WB, Write Back)**：将结果写回寄存器文件——R 型写 ALU 结果；Load 写内存数据；jal 写 PC+4；同时更新 PC。

注意：所有指令的前两步（IF 和 ID）完全相同，后续步骤根据指令类型不同而分叉。

7.3 取指单元 (Instruction Fetch Unit)

取指阶段需要三个核心部件：

- **指令存储器 (Instruction Memory)**：根据 PC 地址输出对应的 32 位指令。
- **程序计数器 (Program Counter, PC)**：存储当前指令的地址，每次取指后自动更新。
- **加法器 (Adder)**：计算 PC+4，用于顺序执行时更新 PC。

为什么是 PC+4？RISC-V 指令固定为 32 位（4 字节），因此顺序执行时下一条指令地址为 PC+4。

指令寄存器 (Instruction Register, IR)：取指阶段还涉及指令寄存器，用于暂存从指令存储器中取出的指令。在单周期 CPU 中，由于整个指令在一个时钟周期内完成，IR 可以省略（指令存储器的输出直接驱动后续逻辑）；但在多周期和流水线 CPU 中，IR 是必要的。

7.4 各类型指令的数据流 (Data Paths by Instruction Type)

R 型指令数据流（以 `add rd, rs1, rs2` 为例）

1. 取指：从指令存储器读指令，PC ← PC+4。
2. 译码：从寄存器文件读 rs1 和 rs2。
3. 执行：ALU 执行相应运算（由 ALU 控制信号决定），得到结果。
4. 访存：跳过（R 型指令不访存）。
5. 写回：将 ALU 结果写入 rd（RegWrite=1，MemtoReg=00）。

Load 指令数据流（以 `ld x1, 200(x2)` 为例）

1. 取指：读指令，PC ← PC+4。
2. 译码：读 rs1 (x2) 的值；ImmGen 将 12 位偏移量符号扩展为 64 位。
3. 执行：ALU 计算内存地址 = rs1 + Imm（ALUSrc_B=1，使用立即数而非 rs2）。
4. 访存：从数据存储器按地址读出 64 位数据（MemRead=1）。
5. 写回：将读出的内存数据写入 rd（MemtoReg=01，RegWrite=1）。

Store 指令数据流（以 `sd x1, 200(x2)` 为例）

1. 取指：读指令，PC ← PC+4。
2. 译码：读 rs1 (x2) 和 rs2 (x1)；ImmGen 从 S 型字段提取并扩展立即数。
3. 执行：ALU 计算地址 = rs1 + Imm。
4. 访存：将 rs2 的值写入该地址（MemWrite=1）。
5. 写回：无（Store 指令不写寄存器文件）。

分支指令数据流（以 `beq x1，x2，200` 为例）

1. 取指：读指令，同时计算 PC+4 备用。
2. 译码：读 rs1 (x1) 和 rs2 (x2)；ImmGen 提取并符号扩展分支偏移量（SB 型，末位补 0）。
3. 执行：ALU 做减法（rs1 - rs2）并输出 Zero 信号；同时另一加法器计算分支目标地址 = PC + Imm（偏移量已左移 1 位）。
4. 访存：跳过。
5. 写回：若 Zero=1 且 Branch=1，则 PC ← PC+Imm；否则 PC ← PC+4。

BNE 指令扩展（以 `bne rs1，rs2，imm` 为例）

`bne`（不等则跳转）与 `beq` 的区别在于条件取反。硬件实现需要额外的 **BranchN** 控制信号：

- 当 `beq`：Branch=1，BranchN=0；跳转条件为 Zero=1（rs1=rs2）。
- 当 `bne`：Branch=0，BranchN=1；跳转条件为 Zero=0（rs1≠rs2）。
- 最终 PC 更新逻辑：PC ← PC+Imm 当 (Branch AND Zero) OR (BranchN AND NOT Zero)。

电路实现使用两个 AND 门加一个 OR 门，将两种分支条件合并为统一的”是否跳转”信号。

跳转指令数据流（以 `jal x1，label` 为例）

1. 取指：读指令。
2. 译码：ImmGen 提取并扩展 UJ 型立即数（末位补 0）。
3. 执行：计算跳转目标地址 = PC + Imm（偏移量已左移 1 位）；同时 PC+4 作为返回地址。
4. 访存：跳过。
5. 写回：将 PC+4 写入 rd（x1）；PC ← PC+Imm（Jump=1）。

7.5 组合数据通路的原则 (Composing the Elements)

- 第一版数据通路在**一个时钟周期内**完成一条指令（单周期 CPU）。
- 每个功能部件在一个时钟周期内只能执行一次操作。
- 指令存储器和数据存储器**分开**（避免同一周期内读写冲突）。
- 使用**多路选择器 (MUX)** 将不同指令类型的数据流合并到统一数据通路上。

7.6 完整单周期数据通路 (Full Single-Cycle Datapath)

完整数据通路通过多路选择器 (MUX) 将不同指令类型的数据流合并，主要 MUX 的作用：

MUX	控制信号	作用
ALU 的 B 端输入	ALUSrc_B	选择寄存器 rs2（0）还是立即数（1）
写回数据选择	MemtoReg	选择 ALU 结果（00）、内存数据（01）或 PC+4（10，用于 jal）
PC 下一值	Branch/BranchN & Zero, Jump	选择 PC+4（顺序）、分支目标或跳转目标

8 控制器设计 (Controller Design)

8.1 控制器的设计思路 (Design Approach)

控制器设计的核心原则

- 所有控制逻辑均为**组合逻辑 (Combinational Logic)**：等待信号稳定后，数据通路输出正确结果。
- 写信号 (Write Signals) 配合时钟边沿决定何时将结果写入状态单元（寄存器文件、PC、数据存储器）。
- **时钟周期由最长路径决定 (Cycle time = longest path)**：即关键路径决定了整个 CPU 的时钟频率上限。

8.2 控制信号总览 (Control Signals Overview)

单周期 CPU 共有 **7 个主控制信号**（由主译码器产生）加上 **4 个 ALU 操作控制信号**（由 ALU 译码器产生）：

7 个主控制信号分为两组：

- **4 个 MUX 控制信号**：ALUSrc_B、MemtoReg（2 位）、Branch、Jump
- **3 个读写控制信号**：RegWrite、MemWrite、MemRead

信号	类型	作用
ALUSrc_B	MUX 控制	ALU B 端：0= 寄存器，1= 立即数
MemtoReg	MUX 控制	写回源：00=ALU 结果，01= 内存数据，10=PC+4（jal）
Branch	MUX 控制	1= 允许 beq 跳转（需结合 Zero 信号）
Jump	MUX 控制	1= 无条件跳转（jal 指令）
RegWR (RegWrite)	R/W 控制	1= 写入寄存器文件
MemWrite	R/W 控制	1= 写入数据存储器
MemRead	R/W 控制	1= 读取数据存储器（Load 指令）

注意：MemtoReg 在支持 `jal` 指令时需要 2 位（00/01/10），以区分三种写回来源。

8.3 两级译码结构 (Two-Level Decoder)

级别	模块	输入 → 输出
第一级	主译码器 (Main Decoder)	opcode[6:0] → 7 个控制信号 + ALUOp[1:0]
第二级	ALU 译码器 (ALU Decoder)	ALUOp[1:0] + funct3 + funct7 → ALU 控制信号 [2:0]

ALUOp 的含义

ALUOp	含义
00	执行加法（用于 Load/Store 计算地址）
01	执行减法（用于 Branch beq/bne 比较）
10	由 funct7&funct3 决定（R 型指令）
11	由 funct3 决定（I 型算术指令，如 addi）

ALU 控制信号真值表 (ALU Control Truth Table)

ALU 译码器使用 4 位组合字段 `Fun = {funct3, funct7[0]}` 进行操作选择：

ALUOp	funct7	funct3	ALU 控制	操作
00	X	X	010	Add（Load/Store 地址计算）
01	X	X	110	Sub（Branch 比较）
10	0000000	000	010	Add（add）
10	0100000	000	110	Sub（sub）
10	0000000	111	000	AND（and）
10	0000000	110	001	OR（or）
10	0000000	010	111	Slt（slt）
10	0000000	100	011	XOR（xor）
10	0000000	101	100	SRL（srl，逻辑右移）
11	X	000	010	Add（addi）

8.4 各指令的控制信号真值表 (Control Signals Truth Table)

指令类型	ALUSrc	MemtoReg	RegWR	MemWR	Branch	Jump	ALUOp
R-type	0	00	1	0	0	0	10
Load (ld)	1	01	1	0	0	0	00
Store (sd)	1	X	0	1	0	0	00
Branch (beq)	0	X	0	0	1	0	01
Jump (jal)	X	10	1	0	0	1	XX
I-ALU (addi)	1	00	1	0	0	0	11

8.5 控制器 Verilog 实现 (Controller Verilog Implementation)

主控制器 (Main Controller)

注意：RISC-V opcode 的低 2 位固定为 `11`，实际译码通常只用 `inst[6:2]` 的 5 位：

```
`define CPU_ctrl_signals \
    {ALUSrc_B, MemtoReg, RegWR, MemWrite, Branch, Jump, ALUop}

always @* begin
    case(OPcode)   // OPcode = inst[6:2], 5-bit
    5'b01100: CPU_ctrl_signals = {1'b0,2'b00,1'b1,1'b0,1'b0,1'b0,2'b10}; // R-type
    5'b00000: CPU_ctrl_signals = {1'b1,2'b01,1'b1,1'b0,1'b0,1'b0,2'b00}; // load
    5'b01000: CPU_ctrl_signals = {1'b1,2'b00,1'b0,1'b1,1'b0,1'b0,2'b00}; // store
```

```
    5'b11000: CPU_ctrl_signals = {1'b0,2'b00,1'b0,1'b0,1'b1,1'b0,2'b01}; // beq
    5'b11011: CPU_ctrl_signals = {1'b0,2'b10,1'b1,1'b0,1'b0,1'b1,2'b00}; // jal
    5'b00100: CPU_ctrl_signals = {1'b1,2'b00,1'b1,1'b0,1'b0,1'b0,2'b11}; // addi(I-ALU)
    default:  CPU_ctrl_signals = 9'bxxxxxxxxx;
    endcase
end
```

ALU 译码器 (ALU Decoder)

```
assign Fun = {funct3, funct7[5]};   // 4-bit: {funct3[2:0], funct7[5]}

always @* begin
    case(ALUop)
    2'b00: ALU_Control = 3'b010;    // add (计算地址)
    2'b01: ALU_Control = 3'b110;    // sub (比较条件)
    2'b10:
        case(Fun)
        4'b0000: ALU_Control = 3'b010; // add
        4'b0001: ALU_Control = 3'b110; // sub
        4'b1110: ALU_Control = 3'b000; // and
        4'b1100: ALU_Control = 3'b001; // or
        4'b0100: ALU_Control = 3'b111; // slt
        4'b1000: ALU_Control = 3'b011; // xor
        4'b1010: ALU_Control = 3'b100; // srl (逻辑右移)
        default: ALU_Control = 3'bxxx;
        endcase
    2'b11:
        case(funct3)
        3'b000: ALU_Control = 3'b010; // addi
        default: ALU_Control = 3'bxxx;
        endcase
    endcase
end
```

9 单周期实现的性能分析 (Single-Cycle Implementation Performance)

9.1 关键路径分析 (Critical Path Analysis)

在单周期 CPU 中，**时钟周期必须满足最慢指令的需求**。假设各部件延迟如下：

- 指令存储器：200 ps
- ALU / 加法器：200 ps
- 数据存储器：200 ps
- 寄存器文件读/写：各 100 ps

各类指令的关键路径时间：

指令	路径	时间
Load (`ld`)	取指 (200) + 读寄存器 (100) + ALU(200) + 读内存 (200) + 写寄存器 (100)	**800 ps**
Store (`sd`)	取指 (200) + 读寄存器 (100) + ALU(200) + 写内存 (200)	700 ps
R-type	取指 (200) + 读寄存器 (100) + ALU(200) + 写寄存器 (100)	600 ps
Branch (`beq`)	取指 (200) + 读寄存器 (100) + ALU(200)	500 ps
Jump (`jal`)	取指 (200) + 写寄存器 (100)	300 ps

关键路径 (Critical Path) 为 Load 指令，共 800 ps。单周期 CPU 的时钟周期必须设置为 800 ps，时钟频率为 $1/800\,\text{ps} = 1.25\,\text{GHz}$。

9.2 单周期实现的缺陷 (Limitations of Single-Cycle)

单周期实现的主要问题

1. **时钟周期由最慢指令决定**：即使执行简单指令（如 `jal` 只需 300 ps），时钟也必须等待最长指令（800 ps），造成大量时间浪费。
2. **功能部件面积浪费（无法复用）**：每条指令独占所有功能部件。若某指令需要 ALU 执行多次操作（如乘法 `mult`），则 CPU 面积将急剧增加，且每个功能部件在一个时钟周期内只能使用一次。
3. **违反”使常见情况快速”原则 (Making the Common Case Fast)**：绝大多数指令不需要访问数据存储器，却被迫等待最慢的 Load 指令来决定时钟周期。

解决方案：引入**流水线 (Pipelining)** 技术，通过指令级并行显著提升吞吐量。

10 全章总结 (Summary)

本章围绕”如何设计并分析一个处理器”展开，核心脉络如下：

1. **冯·诺依曼结构是现代 CPU 的基础**：存储程序、统一数据/程序存储，CPU 通过取指-译码-执行-访存-写回循环运行。
2. **数据通路由寄存器文件、ALU、存储器和 ImmGen 构成**：不同指令类型通过 MUX 在统一数据通路上复用不同数据路径；beq 和 bne 通过 Branch/BranchN 信号区分跳转条件。
3. **控制器产生控制信号驱动数据通路**：采用两级译码（主译码器 + ALU 译码器）；7 个主控制信号分为 4 个 MUX 控制和 3 个读写控制；所有控制逻辑为纯组合逻辑。
4. **CPU 性能由 IC、CPI、时钟周期三者共同决定**：提升性能须综合考虑算法、编译器、ISA 和微体系结构设计。
5. **Amdahl 定律揭示性能提升的上界**：整体加速比 $= 1/[(1 - F_e) + F_e/S_e]$；即使无限加速改进部分，加速比也不超过 $1/(1 - F_e)$；优化应聚焦于执行时间占比最大的瓶颈。
6. **三堵墙推动多核时代的到来**：功耗墙（$P \propto CV^2 f$）、存储墙（CPU 远快于 DRAM）、ILP 墙（约 20% 可挖掘）使单核性能提升面临天花板，多核并行成为必然趋势。
7. **单周期 CPU 是流水线的基础**：理解单周期设计的结构（关键路径 800ps = Load 指令）和性能局限，是学习流水线 CPU 的前提。

”处理器设计的核心是在正确性、性能和成本之间寻求最佳平衡。”